



GIGO

Computer Science + Machine Learning = Computational Chemistry and discoveries

By: Mark Plowright

Different Levels of Languages used by Computers



Language Levels in a Computer System

1. Machine Language

- Binary (0101 1000 0001) or Hexadecimal

2. Assembly Language

- **What it is:** Human-readable mnemonics for machine instructions, often written in plain text editors like Notepad.
- Close to hardware, with a 1:1 correspondence to binary.
- *Example:* `li $v0, 5` (loads 5 into register `$v0`, a small memory slot in the CPU), `li $v1, 3`, `addi $t2, $v0, $v1`

3. High-Level Programming Languages

- **What it is:** Human-readable languages used for general-purpose software and scripts, typically developed in Integrated Development Environments (IDEs).
- **Examples:** Python, JavaScript, Java, C++
- Abstracts away hardware details, making them much easier to write.
- *Example:* `def calculate_y(x): y = 4 * x + 3 return y`

Coding Languages vs Machine Learning Models



Coding Models

- You program the computer with a specific function.
- The computer then uses its Arithmetic Logic Unit (ALU) to solve any input by executing that equation quickly, returning the desired output.

Machine Learning (ML)

- You provide the system with data.
- A model, trained on this historical data, learns the underlying function independently, without explicit programming.
- This process relies on metadata and algorithms.

ALU Arithmetic Logic Unit



The Arithmetic Logic Unit (ALU) executes two primary functions, emphasizing the crucial role of storage and memory:

1. **Arithmetic operations:** These encompass fundamental calculations such as addition, subtraction, multiplication, and division, along with incrementing and decrementing values.
 2. **Logic operations:** This category includes operations such as AND, OR, NOT, XOR, and various comparative evaluations (e.g., equality, greater than, less than).
- The ALU receives two binary numbers (operands) and a control signal, which dictates the specific operation to be performed.
 - These inputs are processed via a data pipeline.
 - The output comprises the operation's result and several status flags, including the zero flag, carry flag, and overflow flag.
 - These flags serve as triggers, initiating specific actions contingent upon the result's bit values (e.g., a zero flag is set if the result is 0, or a carry-out flag is generated for arithmetic operations).

How does a Computer handle your calculations



- Computers operate on the principle of "Garbage in, Garbage Out" (GIGO); their intelligence is limited by the quality of the input.
- All numbers are converted into binary (e.g., 2 becomes 00000010), where specific bits are "on" and the rest are "off."
- For an expression like $4x + 3$, a computer breaks it down into basic electrical decisions (1s and 0s) to perform multiplication, addition, and arrive at the final answer.

Python Example:

```
def calculate_y(x): # Defines a function called calculate_y that takes 'x' as input
```

```
    y = 4 * x + 3
```

```
    return y # Returns the calculated value of 'y'
```

- In this function, 'x' is the parameter (input value), and 'return y' provides the output.
- To get a result, you call the function with an input value: `result = calculate_y(2)`
- The computer uses its Arithmetic Logic Unit (ALU) to calculate the result of mathematical expressions like $4 * x + 3$.

Machine Learning:



Larger data sets such as Dipole Moment we can train Machine Learning Models to find information we need about a calculation.

- **Dipole moment** is a complex property that depends on:
Atomic types, Molecular geometry (3D positions), Electron distribution (quantum effects!)
- A generic ML model like a **Random Forest**: “*which I would normally use*” for patterns only with data...
Doesn't know chemistry, Can't interpret atoms, bonds, or 3D structure, Can't generalize unless trained on very similar data **with proper input encoding** (meaning training the model)

Use (or train) a model that understands chemical structure:

- **SchNet, DimeNet, PhysNet** (neural networks for molecular data)
- **sGDML** (Gaussian process trained on forces).



How would ML learn or work?

A functional machine learning model was developed in Python, capable of processing Gaussian dipole vector data (x, y, z) and predicting dipole magnitude to indicate molecular polarity. The development involved:

- **Step 1:** Creating a Python script for model importation.
- **Step 2:** Incorporating necessary dependencies and modules (e.g., machine learning libraries, CSV file handling, and other relevant files) into the script.
- **Step 3:** Training the model using data from a CSV file.

Project1 C:\Users\mlppl\PycharmProjects\python

al Libraries

hes and Consoles

```
1
2 import pandas as pd
3 import re
4 import matplotlib.pyplot as plt
5 from sklearn.ensemble import RandomForestRegressor
6 from sklearn.model_selection import train_test_split
7 from sklearn.metrics import mean_squared_error, r2_score
8
9 # === STEP 1: Parse Dipole Moments from Gaussian Log Files ===
10
11 # File paths (change these to your local file paths in PyCharm)
12 log_files = {
13     "M-PIMM-DM": "M-PIMM-DM.log",
14     "L-PIMM-DM": "L-PIMM-DM.log",
15     "B-PIMM-DM": "B-PIMM-DM.log"
16 }
17
18 # Regex pattern to extract dipole moment data
```

Local x Git Bash x + v

SKTOP-6EBKCOV MINGW64 ~/PycharmProjects/pythonProject1

Install pandas matplotlib scikit-learn

Requirement already satisfied: pandas in c:\users\mlppl\pycharmprojects\pythonproject1\venv\lib\site-packages (2.2.3)

Installing matplotlib

Installing matplotlib-3.9.4-cp39-cp39-win_amd64.whl (7.8 MB)

| 7.8 MB 2.2 MB/s

Installing scikit-learn

Installing scikit_learn-1.6.1-cp39-cp39-win_amd64.whl (11.2 MB)

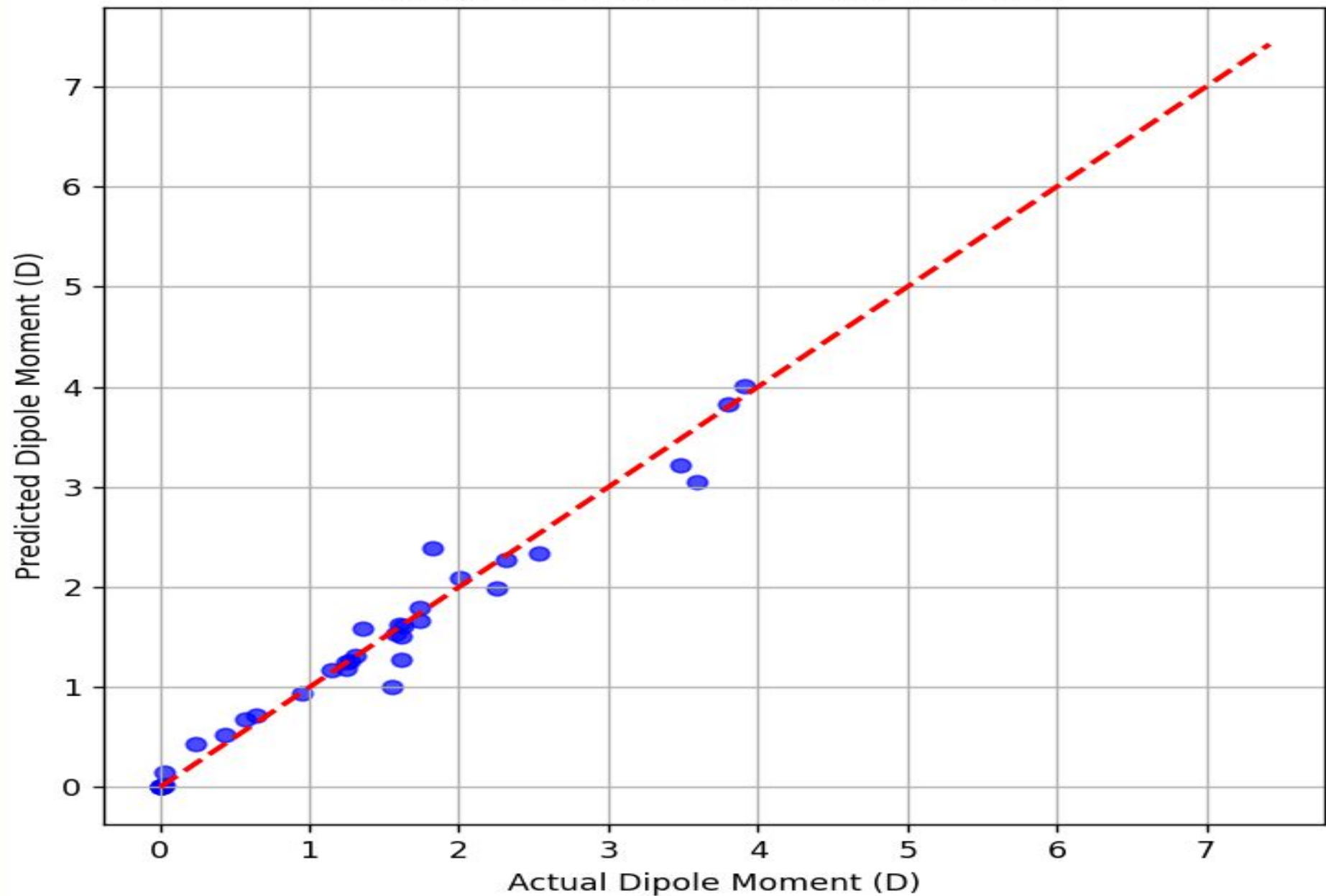
| 11.2 MB ...



Figure 1



Actual vs Predicted Dipole Moments



s > dipole_prediction_pipeline.py



dipole_prediction_pipeline ▾

```
1
2 import pandas as pd
3 import re
4 import matplotlib.pyplot as plt
5 from sklearn.ensemble import RandomForestRegressor
6 from sklearn.model_selection import train_test_split
7 from sklearn.metrics import mean_squared_error, r2_score
8
9 # === STEP 1: Parse Dipole Moments from Gaussian Log Files ===
10
11 # File paths (change these to your local file paths in PyCharm)
12 log_files = {
13     "M-PIMM-DM": "M-PIMM-DM.log",
14     "L-PIMM-DM": "L-PIMM-DM.log",
15     "B-PIMM-DM": "B-PIMM-DM.log"
16 }
17
18 # Regex pattern to extract dipole moment data
```

pipeline ×

pl\PycharmProjects\pythonProject1\venv\Scripts\python.exe C:/Users/mlppl/Downloads/dipole_prediction_pipeline.py

Error: 0.0347

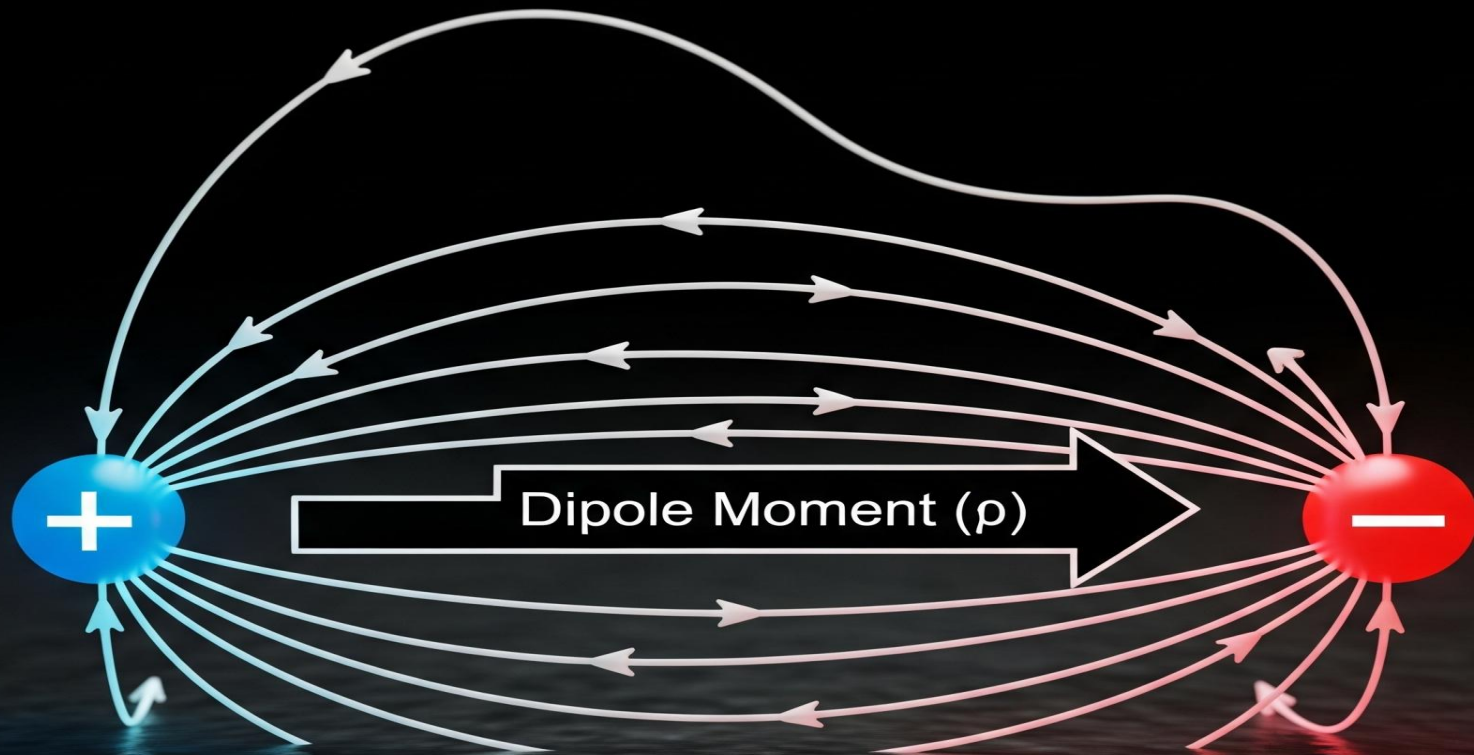
9726



Model Results.

Mean Squared Error (MSE) and R-squared (R^2)

- **MSE ≈ 0.03 :** This indicates a very low average error for the model.
 - MSE is computed by averaging the squared differences between the actual and predicted values.
 - Squaring the errors ensures that positive and negative errors do not cancel each other out, and it penalizes larger errors more significantly.
- **$R^2 \approx 0.97$:** This signifies that the model explains approximately 97% of the variation in dipole moments.
 - R^2 quantifies how much better the model performs compared to simply predicting the average value in every instance.
 - It is calculated as 1 minus the ratio of the model's error to the error obtained from merely predicting the average.



Questions?